04 Manual tecnico SaborExpress

Equipo de desarrollo SaborExpress

Filiación institucional y autores individuales: por confirmar

Estudio académico de ingeniería de software y documentación

14 de octubre de 2026

Manual técnico y operación del sistema

Proyecto: SaborExpress | **Fecha de corte:** 14 de octubre de 2026 | **Versión:** 2.0 | **Tipo:** Documento técnico y académico

Estado de la evidencia: inspección documental y estática del código. Las pruebas de ejecución y los datos institucionales no aportados no se presumen realizados.

1. Visión operativa

La solución consta de repositorios independientes para API, web, móvil y documentación. Las fronteras entre interfaz, lógica y persistencia siguen una orientación feature-first: controladores delegan a servicios y repositorios; EF Core mantiene las entidades; Next y Expo presentan flujos por rol.

2. Referencia de carpetas y módulos

Repositorio	Ruta	Propósito
Back	Program.cs	Bootstrap HTTP, auth, SignalR, servicios hospedados
Back	Configuration/	Dependencias, CORS, JWT, DB y Swagger
Back	Controllers/	Endpoints
Back	Modules/*/Services	Reglas de aplicación
Back	Modules/*/Repositories	Acceso de dominio a BD
Back	Data/Configurations	EF Fluent API
Back	Migrations/	Evolución relacional
Web	src/app	Rutas Next.js
Web	src/features	Servicios/hooks por feature
Movil	src/features	Vistas/servicios/hooks
Movil	app.json y eas.json	Configuración Expo y builds

3. Modelo de autenticación y autorización

El backend registra JWT Bearer, políticas de autorización y usuarios/roles. El rol debe comprobarse en servidor y complementarse con acceso a recurso concreto. ActiveShiftHandler

introduce comprobaciones de turno en acciones designadas. Para login se identifican casos de bloqueo por intentos fallidos, correo sin confirmar y activación inicial. Asegurar expiración/revocación de sesiones.

4. Integraciones de datos

AppDbContext define DbSets por módulo y ejecuta configuraciones EF desde el ensamblado. DatabaseConfiguration.cs elige proveedor por valor literal; SQL Server, MySQL y PostgreSQL tienen diferencias semánticas, por lo que las migraciones requieren validación propia en el proveedor activo. Seeders iniciales: permisos, rol cliente y gerente inicial. Antes de usarlos en producción, revisar cuentas por defecto, reejecución y seguridad de secretos.

5. Integración de cobros

PaymentsController distingue pago manual, inicio Wompi para cliente, inicio QR de cajero, consulta por orden, sincronización y webhook externo. WompiPendingPaymentsSyncService aparece registrado en Program.cs. Operacionalmente, las referencias y transacciones deben reconciliarse sin duplicaciones. El body del webhook aparece registrado por el controlador; antes de operación real, evaluar si el log podría contener información sensible y aplicar filtrado apropiado.

6. Notificaciones y tiempo real

El Hub /hubs/notifications es parte de la API, no un microservicio aislado. Clientes web y móvil declaran @microsoft/signalr; se requiere probar negociación, reconexión, permisos de

grupos/canales y coherencia con consulta REST. No se afirma una garantía de mensajes exactamente una vez.

7. Variables sensibles y despliegue

No guardar valores de ConnectionStrings, claves JWT, tokens Wompi, conexiones SMTP ni credenciales de servicio en el repositorio. Recomendación: appsettings.json sin secretos, .env.example redactado, pipeline con secret store, HTTPS, rotación de claves y administración de dominios permitidos.

8. Logs, monitoreo y soporte

Registrar ID de correlación, ruta, estado HTTP, duración y evento de pago sin datos financieros; separar logs de auditoría. Métricas a implementar: error rate, latencia p95, fallas de transacción, colas de pago pendiente, SignalR reconnections, fallos de migración, recuperación tras backup. Registrar incidentes con versión/commit.

9. Seguridad y mantenimiento

Área	Control recomendado	Evidencia a entregar
JWT/roles	Tests 401/403, acceso por objeto	Matriz negativa
Wompi	Validación firma, replay, duplicados	Sandbox con IDs censurados
Base de datos	Backup/restore y migración idempotente	Log de restauración
DevOps	Escaneo secrets/deps y CI	Reporte de workflow
Frontend	A11y, validación y errores	Checklist por pantalla
Puesta en producción	HTTPS, CORS restringido	Config redactada revisada

10. Procedimiento de liberación sugerido

Crear rama → revisión por pares → compilar API/web/móvil → ejecutar pruebas unitarias/integración → verificar migrations dry run y sandbox de pagos → respaldar →

desplegar en ventana acordada → smoke tests → monitorear → cerrar acta. Rollback debe restaurar binarios y estado coherente, no solo ejecutar git revert.

8. Inventario detallado de módulos del backend

Los directorios siguientes fueron identificados bajo Modules/ en el backend. La columna de funcionamiento describe su responsabilidad inferida por el nombre y las rutas inspeccionadas; funciones no verificadas deben validarse en servicio/controlador.

M01. Addresses

Responsabilidad funcional: Direcciones del cliente. **Ruta de código:** SaborExpressBack/Modules/Addresses/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** validación de propiedad y campos de geocodificación. Documentar transacción y error con correlación sin copiar información personal en los logs.

M02. Auth

Responsabilidad funcional: Inicio de sesión, tokens, recuperación y activación. **Ruta de código:** SaborExpressBack/Modules/Auth/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** protección de credenciales, bloqueo y revocación. Documentar transacción y error con correlación sin copiar información personal en los logs.

M03. Branches

Responsabilidad funcional: Sedes y consultas geográficas. **Ruta de código:** SaborExpressBack/Modules/Branches/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** aislamiento entre sucursales y coordenadas. Documentar transacción y error con correlación sin copiar información personal en los logs.

M04. Categories

Responsabilidad funcional: Clasificación del catálogo. **Ruta de código:** SaborExpressBack/Modules/Categories/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** consistencia de categoría al editar/eliminar. Documentar transacción y error con correlación sin copiar información personal en los logs.

M05. Configurations

Responsabilidad funcional: Ajustes operativos por sede y usuario. **Ruta de código:** SaborExpressBack/Modules/Configurations/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** evitar exposición de configuración entre sedes. Documentar transacción y error con correlación sin copiar información personal en los logs.

M06. Customers

Responsabilidad funcional: Perfil de compradores. **Ruta de código:** SaborExpressBack/Modules/Customers/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** identidad, privacidad y vinculación a usuario. Documentar transacción y error con correlación sin copiar información personal en los logs.

M07. DailyMenu

Responsabilidad funcional: Disponibilidad y menú diario. **Ruta de código:** SaborExpressBack/Modules/DailyMenu/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** sincronizar productos, fecha y sede. Documentar transacción y error con correlación sin copiar información personal en los logs.

M08. Deliveries

Responsabilidad funcional: Asignación, estado y calificación de reparto. **Ruta de código:** SaborExpressBack/Modules/Deliveries/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** autorización por conductor/cliente/sede. Documentar transacción y error con correlación sin copiar información personal en los logs.

M09. EmployeeSchedules

Responsabilidad funcional: Turnos y cobertura del personal. **Ruta de código:** SaborExpressBack/Modules/EmployeeSchedules/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** autoridad de turno para acciones protegidas. Documentar transacción y error con correlación sin copiar información personal en los logs.

M10. Employees

Responsabilidad funcional: Registro y perfiles de personal. **Ruta de código:** SaborExpressBack/Modules/Employees/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** protección de documento, CV y teléfono. Documentar transacción y error con correlación sin copiar información personal en los logs.

M11. Invoices

Responsabilidad funcional: Comprobantes y facturas. **Ruta de código:** SaborExpressBack/Modules/Invoices/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** integridad financiera y folios. Documentar transacción y error con correlación sin copiar información personal en los logs.

M12. Notifications

Responsabilidad funcional: Notificaciones y hub. **Ruta de código:** SaborExpressBack/Modules/Notifications/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** canales autorizados y reconexión. Documentar transacción y error con correlación sin copiar información personal en los logs.

M13. Orders

Responsabilidad funcional: Pedidos, detalles e historia. **Ruta de código:** SaborExpressBack/Modules/Orders/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** cálculo de totales y transiciones. Documentar transacción y error con correlación sin copiar información personal en los logs.

M14. Payments

Responsabilidad funcional: Cobros, Wompi y devoluciones. **Ruta de código:** SaborExpressBack/Modules/Payments/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** checksum, referencias y idempotencia. Documentar transacción y error con correlación sin copiar información personal en los logs.

M15. Permissions

Responsabilidad funcional: Permisos nominales. **Ruta de código:** SaborExpressBack/Modules/Permissions/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** denegación de privilegios no otorgados. Documentar transacción y error con correlación sin copiar información personal en los logs.

M16. Products

Responsabilidad funcional: Catálogo por categoría y sede. **Ruta de código:** SaborExpressBack/Modules/Products/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** precio global/local y desactivación. Documentar transacción y error con correlación sin copiar información personal en los logs.

M17. Reviews

Responsabilidad funcional: Opiniones del servicio. **Ruta de código:** SaborExpressBack/Modules/Reviews/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** derecho del autor y moderación. Documentar transacción y error con correlación sin copiar información personal en los logs.

M18. RolePermissions

Responsabilidad funcional: Relación roles-permisos. **Ruta de código:** SaborExpressBack/Modules/RolePermissions/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** elevación no autorizada. Documentar transacción y error con correlación sin copiar información personal en los logs.

M19. Roles

Responsabilidad funcional: Catálogo de roles. **Ruta de código:** SaborExpressBack/Modules/Roles/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** jerarquía y asignación. Documentar transacción y error con correlación sin copiar información personal en los logs.

M20. Tables

Responsabilidad funcional: Mesas y ocupación. **Ruta de código:** SaborExpressBack/Modules/Tables/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** concurrencia y sede de la mesa. Documentar transacción y error con correlación sin copiar información personal en los logs.

M21. UserPreferences

Responsabilidad funcional: Idioma y preferencias. **Ruta de código:** SaborExpressBack/Modules/UserPreferences/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** perfiles propios y coherencia del locale. Documentar transacción y error con correlación sin copiar información personal en los logs.

M22. UsersRoles

Responsabilidad funcional: Vinculación de usuarios y roles. **Ruta de código:** SaborExpressBack/Modules/UsersRoles/.

Patrón de integración: controlador HTTP o servicio correspondiente → interfaces → servicio de aplicación → repositorios EF Core. Verificar implementación exacta y evitar llamadas directas a entidades desde la vista.

Prueba principal: aislar la característica con datos ficticios, comprobar resultado permitido y excepción de seguridad. **Riesgo prioritario:** actualización y revocación efectiva. Documentar transacción y error con correlación sin copiar información personal en los logs.

9. Contratos, autenticación y ciclo de petición

ASP.NET Core Program.cs: registro de controladores con ApiExceptionFilter y conversión de enum a cadenas, CORS, DbContext, JWT, autenticación, autorización, Swagger, SignalR, hosted services y migración/semillas. Verificar orden de middleware: UseCors, UseAuthentication, UseAuthorization, MapControllers, MapHub.

Token: esquema HTTP Authorization: Bearer; se requiere comprobar firma, emisor, audiencia, expiración y manejo de refresh. No se debe copiar un token real a documentación o capturas. Los clientes deben invalidar sesión cuando el servidor rechaza un token y reducir almacenamiento de secretos en el dispositivo.

Roles: etiquetas Authorize(Roles=...) y ActiveShift son un nivel inicial; la autorización de objeto exige recuperar sede/propietario desde BD y compararlos con el actor. En DeliveriesController hay comprobaciones por repartidor y sede explícitas. Debe inspeccionarse el resto de controladores para garantizar paridad.

Códigos HTTP: 200 consulta válida, 201 creación, 204 actualización sin cuerpo, 400 solicitud inválida, 401 sin autenticación, 403 sin permisos, 404 no encontrado, 409 conflicto

cuando aplique, 500 error inesperado. Son convenciones recomendadas; verificar el filtro real antes de fijar un contrato OpenAPI.

10. Persistencia, migraciones y proveedores

AppDbContext.cs.cs identifica entidades y aplica automáticamente configuraciones del ensamblado. DatabaseConfiguration.cs enruta entre MySQL, SqlServer y PostgreSQL mediante DatabaseProvider y cadenas de conexión nombradas. Program.cs invoca MigrateAsync() al arranque y posteriormente PermissionSeeder, ClienteRoleSeeder e InitialManagerSeeder.

Riesgo operativo: ejecutar migración automática en cada réplica y reinicio puede alterar producción inesperadamente. Propuesta profesional: ejecutar migraciones aprobadas en paso único de despliegue con backup previo; comparar estado EF y esquema; ejecutar smoke tests y habilitar tráfico después.

Compatibilidad: las migraciones EF podrían contener sintaxis o tipos exclusivos de un motor; aunque el código declara tres proveedores, solo un ensayo real por cada uno permite declarar soporte efectivo. Elegir oficialmente un motor para producción y documentarlo.

Rendimiento: registrar planes de consulta y medir consultas repetidas para Orders por fecha/sede, Payments por orderId, Deliveries por empleado y Products por branchId; los índices exactos deben evaluarse desde el esquema desplegado, no inventarse.

11. Integración financiera y reconciliación

PaymentsController ofrece wompi/init, wompi/cashier-init, {id}/wompi-sync y un wompi/webhook anónimo a nivel de atributo. En el webhook se lee cuerpo crudo y X-Event-Checksum, y el servicio recibe ambos; la presencia de ese código no confirma validación correcta de la firma.

Proceso recomendado: 1) crear referencia con unicidad; 2) almacenar intento pendiente; 3) iniciar Wompi en sandbox; 4) recibir payload crudo; 5) verificar suma de integridad de acuerdo con entorno y documentación oficial de Wompi; 6) correlacionar referencia/monto/divisa y estado; 7) abrir transacción para actualizar pago y pedido; 8) garantizar procesamiento idempotente; 9) registrar auditoría sin payload completo; 10) recuperar por sincronización si el webhook no llega.

Riesgos: validar evento no equivale a verificar que pertenece al pedido; transacción repetida no debe confirmar dos veces. El timestamp, la referencia y la firma deben contrastarse. Un intento Wompi reciente pendiente aparece en lógica de OrdersController para impedir confirmación manual; ensayar los límites de 30 minutos y carreras concurrentes.

12. Operación, monitoreo y contingencia

Supervisión mínima: solicitudes por segundo, duración p50/p95, tasa de errores 4xx/5xx, latencia DB, cola de notificaciones, desconexiones SignalR, fallos de webhook y tiempo de reconciliación de pagos.

Respuesta a incidentes: identificar incidente y contexto, detener nuevas modificaciones si hay corrupción, salvaguardar logs anonimizados, tomar respaldo, reproducir en sandbox, aplicar corrección mínima, probar regresión, desplegar con aprobación y documentar causa raíz.

Respaldos: seleccionar política de retención, encriptar y restringir acceso. Probar restauración en ambiente aislado con verificación de órdenes, pagos, relaciones, índices y migraciones. Solo después de ensayo se pueden publicar valores de RPO/RTO.

Seguridad operativa: TLS público, rotación de claves, control de origen CORS, dependencia actualizada, no compartir backups por medios inseguros, auditar descargas de CV y proteger URLs de mapas/localización. Aplicar NIST SSDF y OWASP API Security Top 10 como controles de referencia.

13. Mantenimiento y diagnóstico

Síntoma	Comprobación inicial	Acción de contención	Escalamiento
Error de startup DB	variables y conexión, migración fallida	restaurar servicio sin migrar a ciegas	desarrollador EF/DBA
401 generalizado	reloj, emisión JWT, claves	confirmar expiración y credenciales	equipo auth
403 solo una sede	rol, BranchId, políticas	no deshabilitar autorización	backend/operación
Webhook sin confirmar	referencia, checksum, evento sandbox	sincronización controlada	integración pagos
Panel sin novedades	SignalR y GET de reconciliación	reconectar y consultar estado	web/móvil
Cambio de monto incorrecto	DTO, precio de producto, transacción	detener cobros y auditar	lider técnico y caja
Falla de despliegue web	variables NEXT y versión Node	volver a build aprobado	frontend web
Falla de build móvil	versión Expo/React Native y SDK	build anterior conocido	frontend móvil

Fuentes primarias de implementación

- API: https://github.com/yersonct/SaborExpressBack/tree/main
- Web: https://github.com/yersonct/SaborExpressWeb/tree/main
- Móvil: https://github.com/yersonct/SaborExpressMovil/tree/main
- Referencia de presentación: https://github.com/jsgh1/SRD-v1/tree/dev/docs/docs-SRD

Control documental: documento elaborado a partir del estado observado en octubre de 2026. Antes de entregar, verificar el commit exacto, responsable, revisión del instructor y cualquier cambio posterior del código.